

From Gaussian AR(1) Belief Propagation to General Markov-Chain Diffusion Scores

Exact BP, continuous-message bottlenecks, and Gaussian-message approximation error

Prepared for the thesis project discussion

July 2026

Abstract

This note generalizes the Gaussian AR(1) belief-propagation derivation to arbitrary clean Markov-chain priors observed through coordinatewise Ornstein–Uhlenbeck noising. The main point is structural: Gaussianity is not needed for BP exactness. If the clean prior is a chain and the likelihood factorizes sitewise, then the posterior is again a chain, hence a tree factor graph, and sum-product BP computes exact posterior marginals. The diffusion score is then obtained from the exact identity

$$\nabla_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t \mathbb{E}[a_i \mid x]}{\Delta_t}.$$

Gaussianity only gives a finite-dimensional closure of the continuous BP messages. Outside the Gaussian case, messages are continuous probability functions and must be represented numerically or approximated. We therefore design two algorithms: reference grid BP, and Gaussian message closure carried out directly on the two information parameters. A proposition below shows that the latter is exactly the linear (LMMSE) denoiser, so the error it incurs is the excess error of the best linear estimator over the exact one. The grid is validated in the Gaussian AR(1) case against the exact precision-matrix score, and then used as a numerical proxy for the true score in a Laplace-innovation non-Gaussian chain. The experiments quantify the Gaussian-message approximation error at the posterior-mean and score levels.

Contents

1	Executive summary	3
2	The diffusion setup	3
2.1	Clean chain prior	3
2.2	Coordinatewise OU noising	4
3	Posterior factorization: the posterior remains a chain	4
4	Generic sum-product BP and exactness on trees	5
4.1	Generic factor graph notation	5
4.2	Why BP is exact on trees	6
5	Forward-backward BP for the posterior chain	6
6	The score identity: every algebraic step	7

7	What was special about the Gaussian AR(1) case?	9
7.1	Gaussian clean prior	9
7.2	Gaussian message closure	10
7.3	The key distinction	11
8	The continuous non-Gaussian bottleneck	11
9	Approximation error and score error	11
10	Algorithm 1: reference grid BP	12
10.1	Grid representation	12
10.2	Forward recursion	12
10.3	Backward recursion	13
10.4	Belief, posterior mean, and score	13
10.5	Truncation diagnostic	13
11	Algorithm 2: Gaussian message closure	14
11.1	Closure is the linear denoiser	14
11.2	The discarded grid projection	15
12	Experimental models	15
12.1	Gaussian AR(1): validation model	15
12.2	Laplace-innovation AR(1): non-Gaussian test model	16
13	Error metrics	16
14	Numerical results from the report run	17
14.1	Gaussian validation of grid BP	17
14.2	Laplace grid convergence	19
14.3	Gaussian-message approximation error in the Laplace chain	20
14.4	Example beliefs	23
15	Code explanation	25
15.1	Configuration	25
15.2	Densities and numerical normalization	26
15.3	Transition matrices	26
15.4	Grid BP implementation	26
15.5	Gaussian message closure implementation	27
15.6	The grid-projection ablation	27
15.7	Exact Gaussian score functions	27
15.8	Experiment runners	28
15.9	How to run	28
16	Interpretation and next steps	28
A	Files in the delivered folder	29
B	Source references	29

1 Executive summary

The Gaussian AR(1) calculation was useful, but it was not the most general statement. The correct general statement is:

Markov clean prior + coordinatewise OU noising $\Rightarrow$ posterior chain

so

sum-product BP is exact, because chains are trees.

Then the diffusion score follows from the posterior mean:

$$s_i(x, t) = \partial_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t m_i(x, t)}{\Delta_t}, \quad m_i(x, t) = \mathbb{E}[a_i \mid x].$$

The conceptual difference between the Gaussian and non-Gaussian cases is not exactness. The difference is representation:

Case	BP exact?	Message representation
Gaussian AR(1)	yes	two scalars: precision and information
Finite-state Markov chain	yes	finite vector
Continuous non-Gaussian Markov chain	yes	full function/density

Therefore the computational problem is:

In continuous non-Gaussian chains, exact BP propagates functions.

The Gaussian-message approximation forces those functions to remain Gaussian. Jerome's question is then naturally formalized as:

How large is the error caused by Gaussian message closure?

The code implements the following experimental pipeline:

1. Validate grid BP in the Gaussian AR(1) case, where the exact score is known by linear algebra.
2. In a Laplace-innovation AR(1) chain, estimate grid discretization error by comparing a coarse grid with a fine grid.
3. In the same Laplace chain, compare Gaussian message closure with reference grid BP.
4. Convert all posterior-mean errors into score errors using the exact OU identity.

2 The diffusion setup

2.1 Clean chain prior

Let

$$a_{1:n} = (a_1, \dots, a_n)$$

be clean latent variables. For now take each

$$a_i \in \mathcal{A} \subseteq \mathbb{R},$$

but the same factorization logic works for finite state spaces, vector-valued states, or mixed spaces with the appropriate sums/integrals.

The clean prior is assumed to be Markov along the sequence index:

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^n K_i(a_i \mid a_{i-1}), \quad (1)$$

where μ is the initial density and K_i is a transition density or kernel.

This is the structural assumption. It does not assume Gaussianity. For example, K_i can be Gaussian, Laplace, mixture Gaussian, nonlinear, bounded-support, or discrete.

2.2 Coordinatewise OU noising

At diffusion time $t > 0$, we observe

$$x_i = \alpha_t a_i + \sqrt{\Delta_t} z_i, \quad z_i \sim \mathcal{N}(0, 1), \quad (2)$$

independently over i , with

$$\alpha_t = e^{-t}, \quad \Delta_t = 1 - e^{-2t} = 1 - \alpha_t^2. \quad (3)$$

The one-site likelihood is therefore

$$\ell_i(a_i; x_i) := p_t(x_i \mid a_i) = \mathcal{N}(x_i; \alpha_t a_i, \Delta_t). \quad (4)$$

Written explicitly,

$$\ell_i(a_i; x_i) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp \left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right]. \quad (5)$$

Because the noises z_i are independent, the full likelihood factorizes:

$$p_t(x_{1:n} \mid a_{1:n}) = \prod_{i=1}^n \ell_i(a_i; x_i). \quad (6)$$

3 Posterior factorization: the posterior remains a chain

Proposition 1 (Posterior chain factorization). *Assume the clean prior has the Markov factorization (1) and the likelihood factorizes as in (6). Then the posterior is*

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{1}{Z_t(x)} \mu(a_1) \prod_{i=2}^n K_i(a_i \mid a_{i-1}) \prod_{i=1}^n \ell_i(a_i; x_i), \quad (7)$$

where

$$Z_t(x) = p_t(x_{1:n})$$

is the evidence. Hence the posterior factor graph is a chain with unary observation factors.

Proof. Start from Bayes' rule:

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{p_0(a_{1:n})p_t(x_{1:n} \mid a_{1:n})}{p_t(x_{1:n})}. \quad (8)$$

Now substitute the Markov prior:

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^n K_i(a_i \mid a_{i-1}), \quad (9)$$

and the factorized likelihood:

$$p_t(x_{1:n} \mid a_{1:n}) = \prod_{i=1}^n \ell_i(a_i; x_i). \quad (10)$$

Therefore

$$p_t(a_{1:n} \mid x_{1:n}) = \frac{1}{p_t(x_{1:n})} \left[\mu(a_1) \prod_{i=2}^n K_i(a_i \mid a_{i-1}) \right] \left[\prod_{i=1}^n \ell_i(a_i; x_i) \right] \quad (11)$$

$$= \frac{1}{Z_t(x)} \mu(a_1) \prod_{i=2}^n K_i(a_i \mid a_{i-1}) \prod_{i=1}^n \ell_i(a_i; x_i). \quad (12)$$

Every transition factor touches only adjacent variables (a_{i-1}, a_i) , and every likelihood factor touches only one variable a_i . Thus no loops are created. The posterior graph is still a chain. $\square$

Graphically, the posterior structure is

$$a_1 - a_2 - \cdots - a_n,$$

with one unary likelihood ℓ_i attached to each a_i . This is why the BP exactness argument applies.

4 Generic sum-product BP and exactness on trees

4.1 Generic factor graph notation

Let a distribution factorize as

$$p(a) = \frac{1}{Z} \prod_{f \in F} \psi_f(a_{\partial f}), \quad (13)$$

where F is the set of factors and ∂f is the set of variables adjacent to factor f . Let ∂i be the set of factors adjacent to variable a_i .

Sum-product BP passes two types of messages.

The variable-to-factor message is

$$m_{i \rightarrow f}(a_i) \propto \prod_{g \in \partial i \setminus f} m_{g \rightarrow i}(a_i). \quad (14)$$

The factor-to-variable message is

$$m_{f \rightarrow i}(a_i) \propto \int \psi_f(a_{\partial f}) \prod_{j \in \partial f \setminus i} m_{j \rightarrow f}(a_j) \prod_{j \in \partial f \setminus i} da_j. \quad (15)$$

For discrete variables, the integrals are replaced by sums.

Once messages have converged, or after the two sweeps on a tree, the belief at variable i is

$$b_i(a_i) = \frac{\prod_{f \in \partial i} m_{f \rightarrow i}(a_i)}{\int \prod_{f \in \partial i} m_{f \rightarrow i}(u) du}. \quad (16)$$

4.2 Why BP is exact on trees

On a tree, deleting one edge separates the graph into two disconnected components. Therefore a message sent across an edge can be interpreted exactly as the partial marginal contribution of the whole subtree behind that edge.

For example, suppose a message is sent from a factor-side subtree to a boundary variable a_i . The factor-to-variable update integrates all variables inside that subtree except the boundary variable. Since no loop reconnects that subtree to the rest of the graph, this partial marginal contribution is exact. When all incoming messages arrive at a_i , multiplying them combines disjoint subtrees around a_i . Normalizing gives the exact marginal.

Thus on a tree factor graph,

$$b_i(a_i) = p(a_i) \quad (17)$$

exactly. Since a chain is connected and acyclic, a chain is a tree.

Remark 1. *The reason BP is exact here is the Markov-chain structure of the clean prior over the sequence index plus independent sitewise noising. It is not merely the fact that the forward diffusion process is Markov in diffusion time. Diffusion-time Markovianity and sequence-index Markovianity are different statements.*

5 Forward-backward BP for the posterior chain

For the chain posterior (7), generic BP becomes the forward-backward algorithm.

Define left messages L_i and right messages R_i so that the local likelihood ℓ_i is kept outside the incoming message at site i . Initialize

$$L_1(a_1) = \mu(a_1), \quad R_n(a_n) = 1. \quad (18)$$

Then for $i = 1, \dots, n-1$,

$$L_{i+1}(a_{i+1}) = \int_{\mathcal{A}} K_{i+1}(a_{i+1} | a_i) L_i(a_i) \ell_i(a_i; x_i) da_i. \quad (19)$$

For $i = n, n-1, \dots, 2$,

$$R_{i-1}(a_{i-1}) = \int_{\mathcal{A}} K_i(a_i | a_{i-1}) R_i(a_i) \ell_i(a_i; x_i) da_i. \quad (20)$$

The posterior belief at site i is

$$b_i(a_i) = \frac{L_i(a_i) \ell_i(a_i; x_i) R_i(a_i)}{\int_{\mathcal{A}} L_i(u) \ell_i(u; x_i) R_i(u) du}. \quad (21)$$

Because the posterior graph is a chain,

$$b_i(a_i) = p_t(a_i | x_{1:n}). \quad (22)$$

Therefore the exact posterior mean is

$$m_i(x, t) := \mathbb{E}_t[a_i | x_{1:n}] = \int_{\mathcal{A}} a_i b_i(a_i) da_i. \quad (23)$$

6 The score identity: every algebraic step

This is the key diffusion-specific identity. The noisy marginal is

$$p_t(x_{1:n}) = \int_{\mathcal{A}^n} p_0(a_{1:n}) \prod_{j=1}^n \mathcal{N}(x_j; \alpha_t a_j, \Delta_t) da_{1:n}. \quad (24)$$

For compact notation define

$$g_j(x_j | a_j) := \mathcal{N}(x_j; \alpha_t a_j, \Delta_t). \quad (25)$$

Then

$$p_t(x) = \int p_0(a) \prod_{j=1}^n g_j(x_j | a_j) da. \quad (26)$$

We compute the i -th score component:

$$s_i(x, t) = \partial_{x_i} \log p_t(x). \quad (27)$$

Using

$$\partial_{x_i} \log p_t(x) = \frac{\partial_{x_i} p_t(x)}{p_t(x)}, \quad (28)$$

we first differentiate the marginal density. Under the usual regularity/dominated-convergence conditions that hold for the Gaussian likelihoods used here,

$$\partial_{x_i} p_t(x) = \partial_{x_i} \int p_0(a) \prod_{j=1}^n g_j(x_j | a_j) da \quad (29)$$

$$= \int p_0(a) \partial_{x_i} \left[\prod_{j=1}^n g_j(x_j | a_j) \right] da. \quad (30)$$

Only the factor $g_i(x_i | a_i)$ depends on x_i , so

$$\partial_{x_i} \left[\prod_{j=1}^n g_j(x_j | a_j) \right] = [\partial_{x_i} g_i(x_i | a_i)] \prod_{j \neq i} g_j(x_j | a_j). \quad (31)$$

Now compute the derivative of the Gaussian factor explicitly:

$$g_i(x_i | a_i) = \frac{1}{\sqrt{2\pi\Delta_t}} \exp \left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t} \right], \quad (32)$$

$$\log g_i(x_i | a_i) = -\frac{1}{2} \log(2\pi\Delta_t) - \frac{(x_i - \alpha_t a_i)^2}{2\Delta_t}, \quad (33)$$

$$\partial_{x_i} \log g_i(x_i | a_i) = -\frac{1}{2\Delta_t} \partial_{x_i} (x_i - \alpha_t a_i)^2, \quad (34)$$

$$= -\frac{1}{2\Delta_t} 2(x_i - \alpha_t a_i), \quad (35)$$

$$= -\frac{x_i - \alpha_t a_i}{\Delta_t}. \quad (36)$$

Therefore

$$\partial_{x_i} g_i(x_i | a_i) = g_i(x_i | a_i) \partial_{x_i} \log g_i(x_i | a_i) = -\frac{x_i - \alpha_t a_i}{\Delta_t} g_i(x_i | a_i). \quad (37)$$

Substitute (37) into (30):

$$\partial_{x_i} p_t(x) = \int p_0(a) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} g_i(x_i | a_i) \right] \prod_{j \neq i} g_j(x_j | a_j) da \quad (38)$$

$$= \int p_0(a) \prod_{j=1}^n g_j(x_j | a_j) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da. \quad (39)$$

Divide by $p_t(x)$:

$$\partial_{x_i} \log p_t(x) = \frac{1}{p_t(x)} \int p_0(a) \prod_{j=1}^n g_j(x_j | a_j) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da. \quad (40)$$

Now use Bayes' rule in density form:

$$p_t(a | x) = \frac{p_0(a) \prod_{j=1}^n g_j(x_j | a_j)}{p_t(x)}. \quad (41)$$

Thus

$$\partial_{x_i} \log p_t(x) = \int p_t(a | x) \left[-\frac{x_i - \alpha_t a_i}{\Delta_t} \right] da \quad (42)$$

$$= -\frac{1}{\Delta_t} \int p_t(a | x) (x_i - \alpha_t a_i) da \quad (43)$$

$$= -\frac{1}{\Delta_t} \left[x_i \int p_t(a | x) da - \alpha_t \int a_i p_t(a | x) da \right] \quad (44)$$

$$= -\frac{1}{\Delta_t} [x_i - \alpha_t \mathbb{E}_t[a_i | x]]. \quad (45)$$

Therefore

$$\boxed{s_i(x, t) = \partial_{x_i} \log p_t(x) = -\frac{x_i - \alpha_t m_i(x, t)}{\Delta_t}, \quad m_i(x, t) = \mathbb{E}_t[a_i | x].} \quad (46)$$

Combining (22), (23), and (46) gives the main theorem.

Theorem 1 (Exact score by BP for Markov-chain data under OU noising). *Assume*

$$p_0(a_{1:n}) = \mu(a_1) \prod_{i=2}^n K_i(a_i | a_{i-1})$$

and

$$x_i = \alpha_t a_i + \sqrt{\Delta_t} z_i, \quad z_i \sim \mathcal{N}(0, 1)$$

independently over i . Then $p_t(a_{1:n} | x_{1:n})$ is a chain-structured factor graph. Sum-product BP computes the exact posterior marginal

$$b_i(a_i) = p_t(a_i | x_{1:n}).$$

If

$$m_i^{\text{BP}}(x, t) = \int a_i b_i(a_i) da_i,$$

then

$$m_i^{\text{BP}}(x, t) = \mathbb{E}_t[a_i | x],$$

and the exact diffusion score is

$$s_i(x, t) = -\frac{x_i - \alpha_t m_i^{\text{BP}}(x, t)}{\Delta_t}. \quad (47)$$

In vector form,

$$\nabla_x \log p_t(x) = -\frac{x - \alpha_t m^{\text{BP}}(x, t)}{\Delta_t}. \quad (48)$$

7 What was special about the Gaussian AR(1) case?

7.1 Gaussian clean prior

In the Gaussian AR(1) case,

$$a_1 \sim \mathcal{N}(0, 1), \quad a_i = \rho a_{i-1} + \sqrt{q} \xi_i, \quad q = 1 - \rho^2, \quad (49)$$

with $\xi_i \sim \mathcal{N}(0, 1)$. The transition is

$$K_i(a_i \mid a_{i-1}) = \mathcal{N}(a_i; \rho a_{i-1}, q). \quad (50)$$

The covariance matrix of the clean chain is

$$\Sigma_0(i, j) = \rho^{|i-j|}. \quad (51)$$

The noisy vector is also Gaussian:

$$x = \alpha_t a + \sqrt{\Delta_t} z, \quad a \sim \mathcal{N}(0, \Sigma_0), \quad z \sim \mathcal{N}(0, I), \quad (52)$$

so

$$x \sim \mathcal{N}(0, \Sigma_t), \quad \Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I. \quad (53)$$

For any centered Gaussian density

$$p(x) \propto \exp\left(-\frac{1}{2} x^\top \Sigma^{-1} x\right),$$

we have

$$\nabla_x \log p(x) = -\Sigma^{-1} x. \quad (54)$$

Therefore the exact Gaussian score is

$$s_G(x, t) = -\Sigma_t^{-1} x. \quad (55)$$

The exact Gaussian posterior mean is

$$m_G(x, t) = \alpha_t \Sigma_0 \Sigma_t^{-1} x. \quad (56)$$

One can verify directly that (55) and (46) agree:

$$-\frac{x - \alpha_t m_G}{\Delta_t} = -\frac{x - \alpha_t (\alpha_t \Sigma_0 \Sigma_t^{-1} x)}{\Delta_t} \quad (57)$$

$$= -\frac{[I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}] x}{\Delta_t}. \quad (58)$$

Since

$$\Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I, \quad (59)$$

we have

$$\Delta_t I = \Sigma_t - \alpha_t^2 \Sigma_0. \quad (60)$$

Multiplying on the right by Σ_t^{-1} ,

$$\Delta_t \Sigma_t^{-1} = I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}. \quad (61)$$

Hence

$$-\frac{[I - \alpha_t^2 \Sigma_0 \Sigma_t^{-1}]x}{\Delta_t} = -\frac{\Delta_t \Sigma_t^{-1} x}{\Delta_t} \quad (62)$$

$$= -\Sigma_t^{-1} x. \quad (63)$$

7.2 Gaussian message closure

A Gaussian BP message can be written in information form as

$$m(a) \propto \exp\left(-\frac{1}{2}\lambda a^2 + ha\right), \quad (64)$$

where λ is precision and h is information. Its mean and variance are

$$\mu = \frac{h}{\lambda}, \quad v = \frac{1}{\lambda}. \quad (65)$$

The OU likelihood as a function of a_i is also Gaussian-shaped:

$$\ell_i(a_i; x_i) \propto \exp\left[-\frac{(x_i - \alpha_t a_i)^2}{2\Delta_t}\right] \quad (66)$$

$$\propto \exp\left[-\frac{\alpha_t^2}{2\Delta_t} a_i^2 + \frac{\alpha_t x_i}{\Delta_t} a_i\right]. \quad (67)$$

Thus multiplying a Gaussian message by the likelihood updates information parameters as

$$\tilde{\lambda}_i = \lambda_i + \frac{\alpha_t^2}{\Delta_t}, \quad \tilde{h}_i = h_i + \frac{\alpha_t x_i}{\Delta_t}. \quad (68)$$

The resulting temporary mean and variance are

$$\tilde{\mu}_i = \frac{\tilde{h}_i}{\tilde{\lambda}_i}, \quad \tilde{v}_i = \frac{1}{\tilde{\lambda}_i}. \quad (69)$$

Now apply the Gaussian transition

$$a_{i+1} = \rho a_i + \sqrt{q} \xi_{i+1}. \quad (70)$$

If a_i has temporary distribution $\mathcal{N}(\tilde{\mu}_i, \tilde{v}_i)$, then

$$a_{i+1} \sim \mathcal{N}(\rho \tilde{\mu}_i, q + \rho^2 \tilde{v}_i). \quad (71)$$

Therefore the outgoing left message remains Gaussian with

$$\lambda_{i+1} = \frac{1}{q + \rho^2 \tilde{v}_i}, \quad h_{i+1} = \frac{\rho \tilde{\mu}_i}{q + \rho^2 \tilde{v}_i}. \quad (72)$$

This is the finite-dimensional closure. The message update is exact because products and marginalizations of Gaussian densities remain Gaussian.

7.3 The key distinction

The Gaussian AR(1) case combines two facts:

1. BP is exact because the posterior graph is a chain.
2. Messages are finite-dimensional because the Gaussian family is closed under the BP updates.

The first fact survives for any Markov-chain prior. The second does not.

Therefore:

Gaussianity gives closure, not exactness.

8 The continuous non-Gaussian bottleneck

For a general continuous non-Gaussian transition K_i , the update

$$L_{i+1}(a_{i+1}) = \int K_{i+1}(a_{i+1} | a_i) L_i(a_i) \ell_i(a_i; x_i) da_i \quad (73)$$

maps one function into another function. Thus the message space is infinite-dimensional:

$$L_i, R_i \in L_+^1(\mathbb{R}). \quad (74)$$

This is the real computational issue. Exact BP exists mathematically, but one cannot store arbitrary functions exactly on a computer. We must approximate them.

The simplest hierarchy of approximations is:

Approximation	Message representation	Comment
Grid BP	values on M grid points	reference numerical method
Gaussian closure	mean and variance	tests Gaussian closure error
Gaussian mixture BP	mixture weights, means, variances	handles multimodality
Particle BP	weighted samples	Monte Carlo approximation
Basis expansion	coefficients in basis	Hermite/spline/Fourier style
Neural messages	amortized function/embedding	most flexible, but harder to justify

In this project we focus first on the two cleanest algorithms: grid BP and Gaussian message closure.

9 Approximation error and score error

Let $m_i(x, t) = \mathbb{E}[a_i | x]$ be the exact posterior mean and $\hat{m}_i(x, t)$ an approximation. Define

$$s_i(x, t) = -\frac{x_i - \alpha_t m_i(x, t)}{\Delta_t}, \quad \hat{s}_i(x, t) = -\frac{x_i - \alpha_t \hat{m}_i(x, t)}{\Delta_t}. \quad (75)$$

Then the score error is exactly

$$\hat{s}_i - s_i = -\frac{x_i - \alpha_t \hat{m}_i}{\Delta_t} + \frac{x_i - \alpha_t m_i}{\Delta_t} \quad (76)$$

$$= \frac{-x_i + \alpha_t \hat{m}_i + x_i - \alpha_t m_i}{\Delta_t} \quad (77)$$

$$= \frac{\alpha_t}{\Delta_t} (\hat{m}_i - m_i). \quad (78)$$

Thus

$$\boxed{\widehat{s}_i(x, t) - s_i(x, t) = \frac{\alpha_t}{\Delta_t} [\widehat{m}_i(x, t) - m_i(x, t)]}. \quad (79)$$

This identity is important because it tells us that all approximation errors enter through posterior-mean error. The OU factor

$$\frac{\alpha_t}{\Delta_t} = \frac{e^{-t}}{1 - e^{-2t}} \quad (80)$$

behaves as

$$\frac{e^{-t}}{1 - e^{-2t}} \sim \frac{1}{2t} \quad \text{as } t \downarrow 0. \quad (81)$$

Therefore small-noise times amplify posterior-mean errors strongly.

10 Algorithm 1: reference grid BP

10.1 Grid representation

Choose grid points

$$u_1, \dots, u_M \in [-A, A] \quad (82)$$

and quadrature weights $w_1, \dots, w_M$. In the code, equally spaced points and composite trapezoidal weights are used.

A message $L_i(a)$ is represented by the vector

$$(L_i(u_1), \dots, L_i(u_M)). \quad (83)$$

The transition is represented by the matrix

$$K_{k\ell} = K(u_k \mid u_\ell). \quad (84)$$

The likelihood is represented by

$$\ell_{i,k} = \ell_i(u_k; x_i). \quad (85)$$

10.2 Forward recursion

The continuous update is

$$L_{i+1}(a') = \int K(a' \mid a) L_i(a) \ell_i(a; x_i) da. \quad (86)$$

The grid approximation at $a' = u_k$ is

$$L_{i+1}(u_k) \approx \sum_{\ell=1}^M K(u_k \mid u_\ell) L_i(u_\ell) \ell_i(u_\ell; x_i) w_\ell. \quad (87)$$

In matrix form,

$$L_{i+1} \approx K [(L_i \odot \ell_i) \odot w], \quad (88)$$

where $\odot$ denotes componentwise multiplication.

10.3 Backward recursion

The continuous update is

$$R_{i-1}(a) = \int K(a' | a) R_i(a') \ell_i(a'; x_i) da'. \quad (89)$$

The grid approximation at $a = u_\ell$ is

$$R_{i-1}(u_\ell) \approx \sum_{k=1}^M K(u_k | u_\ell) R_i(u_k) \ell_i(u_k; x_i) w_k. \quad (90)$$

In matrix form,

$$R_{i-1} \approx K^\top [(R_i \odot \ell_i) \odot w]. \quad (91)$$

10.4 Belief, posterior mean, and score

The belief is

$$b_i(u_k) = \frac{L_i(u_k) \ell_i(u_k; x_i) R_i(u_k)}{\sum_{\ell=1}^M L_i(u_\ell) \ell_i(u_\ell; x_i) R_i(u_\ell) w_\ell}. \quad (92)$$

The posterior mean is

$$m_i^{\text{grid}} = \sum_{k=1}^M u_k b_i(u_k) w_k. \quad (93)$$

The grid score is

$$s_i^{\text{grid}} = -\frac{x_i - \alpha_t m_i^{\text{grid}}}{\Delta_t}. \quad (94)$$

The cost is roughly $O(nM^2)$, because each forward/backward step applies an $M \times M$ transition matrix.

10.5 Truncation diagnostic

Grid BP restricts the state space to $[-A, A]$, and nothing in the recursion detects when that interval is too narrow: the messages are renormalized at each step, so mass that should have escaped is quietly redistributed over the retained support. Since the algorithm is used here as the *reference* against which everything else is measured, this needs an explicit check rather than a plausibility argument.

The forward messages and the beliefs are genuine probability densities — L_i is the filtered law $p(a_i | x_1, \dots, x_{i-1})$ and b_i is the posterior marginal — so after normalization we monitor the mass they place in the outermost cells,

$$\beta = \sum_{k \leq K_{\text{edge}}} b(u_k) w_k + \sum_{k > M - K_{\text{edge}}} b(u_k) w_k, \quad (95)$$

and abort if β exceeds a tolerance. Backward messages are excluded: R_i is a likelihood $p(x_{i+1}, \dots, x_n | a_i)$ rather than a density, and it is legitimately almost flat at large t , so a boundary-mass criterion does not apply to it.

For the settings of Section 14 the worst value observed over the whole sweep is $\beta = 5.3 \times 10^{-10}$, three orders of magnitude inside the reference accuracy, so the truncation is not contributing to any of the reported errors.

11 Algorithm 2: Gaussian message closure

Gaussian message closure forces every message to be Gaussian:

$$L_i(a) \approx \mathcal{N}(a; \mu_i^L, v_i^L), \quad R_i(a) \approx \mathcal{N}(a; \mu_i^R, v_i^R). \quad (96)$$

The essential point is that *no grid is involved*. A Gaussian is exactly determined by two scalars, so the update can and must be carried out on those two scalars directly. We work in information form,

$$m(a) \propto \exp\left(-\frac{1}{2}\lambda a^2 + ha\right), \quad \text{mean} = \frac{h}{\lambda}, \quad \text{variance} = \frac{1}{\lambda}, \quad (97)$$

which is the finite-dimensional closure already derived in Section 7.2.

Forward step. Multiplying by the local likelihood updates the parameters as in (68),

$$\tilde{\lambda}_i = \lambda_i^L + \frac{\alpha_t^2}{\Delta_t}, \quad \tilde{h}_i = h_i^L + \frac{\alpha_t x_i}{\Delta_t}, \quad \tilde{\mu}_i = \frac{\tilde{h}_i}{\tilde{\lambda}_i}, \quad \tilde{v}_i = \frac{1}{\tilde{\lambda}_i}. \quad (98)$$

Pushing through the transition $a_{i+1} = \rho a_i + \varepsilon_{i+1}$ gives

$$\lambda_{i+1}^L = \frac{1}{q + \rho^2 \tilde{v}_i}, \quad h_{i+1}^L = \frac{\rho \tilde{\mu}_i}{q + \rho^2 \tilde{v}_i}. \quad (99)$$

Backward step. Since $R_{i-1}(a) = \int K(a' | a) R_i(a') \ell_i(a'; x_i) da' \propto \mathcal{N}(a; \tilde{\mu}_i/\rho, (q + \tilde{v}_i)/\rho^2)$, the information-form update is

$$\lambda_{i-1}^R = \frac{\rho^2}{q + \tilde{v}_i}, \quad h_{i-1}^R = \frac{\rho \tilde{\mu}_i}{q + \tilde{v}_i}, \quad (100)$$

written this way so that $\rho \rightarrow 0$ remains well behaved. The flat terminal message is represented exactly by $\lambda_n^R = 0$, $h_n^R = 0$, which is the correct encoding of infinite variance; this is the step at which a grid representation would fail.

Belief. Information parameters simply add:

$$\lambda_i = \lambda_i^L + \lambda_i^R + \frac{\alpha_t^2}{\Delta_t}, \quad h_i = h_i^L + h_i^R + \frac{\alpha_t x_i}{\Delta_t}, \quad m_i^{\text{GMsg}} = \frac{h_i}{\lambda_i}, \quad (101)$$

and the approximate score is

$$s_i^{\text{GMsg}} = -\frac{x_i - \alpha_t m_i^{\text{GMsg}}}{\Delta_t}. \quad (102)$$

In the Gaussian AR(1) case this recursion is exact, and numerically it reproduces the precision-matrix score to machine precision at every diffusion time. In a non-Gaussian chain it is an approximation, and the next proposition identifies exactly which one.

11.1 Closure is the linear denoiser

Proposition 2 (Gaussian closure is blind to non-Gaussian innovations). *Let the clean chain be $a_{i+1} = \rho a_i + \varepsilon_{i+1}$ with i.i.d. innovations satisfying $\mathbb{E}[\varepsilon] = 0$ and $\text{Var}(\varepsilon) = q$, and otherwise arbitrary. Then the Gaussian message closure above produces the same posterior means as it would for a Gaussian chain with the same ρ and q ; that is,*

$$m^{\text{GMsg}}(x, t) = \alpha_t \Sigma_0 \Sigma_t^{-1} x = m_G(x, t), \quad (103)$$

the linear minimum mean-square-error (LMMSE) estimator.

Proof. Both message operations preserve Gaussianity of the *representation* by fiat, so the only question is which moments the updates use. Multiplication by the likelihood is a product of two Gaussians and is exact. For the transition step, let the incoming message be $\mathcal{N}(\tilde{\mu}, \tilde{v})$. The exact outgoing function is the density of $\rho A + \varepsilon$ with $A \sim \mathcal{N}(\tilde{\mu}, \tilde{v})$ independent of ε . Whatever the law of ε , independence gives

$$\mathbb{E}[\rho A + \varepsilon] = \rho \tilde{\mu}, \quad \text{Var}(\rho A + \varepsilon) = \rho^2 \tilde{v} + q. \quad (104)$$

Moment matching retains precisely these two numbers and discards everything else, so the outgoing parameters are (99) irrespective of the shape of ε . The same argument applies to the backward step. Hence every message, and therefore every belief, coincides with the one obtained from the Gaussian chain with matching ρ and q ; for that chain BP is exact and returns $\alpha_t \Sigma_0 \Sigma_t^{-1} x$, which is the LMMSE estimator because the AR(1) second-order structure $\text{Cov}(a_i, a_{i-k}) = \rho^k$ is shared by both models. $\square$

This reframes the experiments of Section 14. The quantity we measure is not a numerical approximation error at all: it is the excess error of the *best possible linear denoiser* relative to the exact one. Equivalently, it is exactly the amount of nonlinearity that a learned score has to supply in order to beat a linear model. The column `closure_vs_lmmse_rel_gap_mean` in the output CSV verifies Proposition 2 numerically; it sits at 10^{-16} at every diffusion time.

11.2 The discarded grid projection

An earlier version of this note implemented the closure differently: each message was stored as $\mathcal{N}(\mu, v)$ but was *evaluated on the grid*, updated there, and moment-matched back. That algorithm is retained in the code as `gaussian_projected_bp_grid` and is reported as an ablation, because its failure mode is instructive.

The terminal backward message is flat. Represented on $[-A, A]$ and moment-matched, a constant function returns variance $(2A)^2/12$ rather than infinity, so the projection injects a spurious precision of order $3/A^2$ into the backward pass. The damage grows with diffusion time, because $\alpha_t \rightarrow 0$ flattens every message and so progressively more of them are fitted to the width of the truncation rather than to their own shape. In the Gaussian case, where the answer should be exact, the resulting relative score error reaches 1.8×10^{-1} at $t = 2.4$ — see Figure 2. This is the origin of the anomalous growth with t , and of the non-monotonic behaviour, in the first version of this report.

The lesson is worth stating plainly: a Gaussian approximation should never be routed through a grid. Doing so combines the bias of the Gaussian assumption with a truncation error that the Gaussian assumption was introduced to avoid.

12 Experimental models

12.1 Gaussian AR(1): validation model

The validation model is

$$a_1 \sim \mathcal{N}(0, 1), \quad a_i = \rho a_{i-1} + \sqrt{1 - \rho^2} \xi_i, \quad \xi_i \sim \mathcal{N}(0, 1). \quad (105)$$

This gives the exact precision-matrix score (55). Therefore we can directly evaluate the grid discretization error:

$$\text{GridErr}_G(t, M) = \frac{\|s_{\text{grid}}^G(x, t) - s_{\text{matrix}}^G(x, t)\|_2}{\|s_{\text{matrix}}^G(x, t)\|_2}. \quad (106)$$

12.2 Laplace-innovation AR(1): non-Gaussian test model

The non-Gaussian model is

$$a_1 \sim \mathcal{N}(0, 1), \quad a_i = \rho a_{i-1} + \varepsilon_i, \quad (107)$$

where

$$\varepsilon_i \sim \text{Laplace}(0, b). \quad (108)$$

The Laplace density is

$$p(\varepsilon) = \frac{1}{2b} \exp\left(-\frac{|\varepsilon|}{b}\right). \quad (109)$$

A $\text{Laplace}(0, b)$ variable has variance $2b^2$. We choose

$$b = \sqrt{\frac{1 - \rho^2}{2}} \quad (110)$$

so

$$\text{Var}(\varepsilon_i) = 2b^2 = 1 - \rho^2. \quad (111)$$

Thus if $\text{Var}(a_{i-1}) = 1$, then

$$\text{Var}(a_i) = \text{Var}(\rho a_{i-1} + \varepsilon_i) \quad (112)$$

$$= \rho^2 \text{Var}(a_{i-1}) + \text{Var}(\varepsilon_i) \quad (113)$$

$$= \rho^2 + (1 - \rho^2) \quad (114)$$

$$= 1. \quad (115)$$

Also, since ε_i is independent of the past,

$$\text{Cov}(a_i, a_{i-k}) = \rho^k \text{Var}(a_{i-k}) = \rho^k. \quad (116)$$

So the second-order covariance structure is still AR(1), but the distribution is non-Gaussian. This makes it a clean model for testing what Gaussian message closure loses beyond covariance information.

In this model there is no closed-form precision-matrix score. We therefore use fine-grid BP as a numerical proxy for the true continuous BP score.

13 Error metrics

For posterior means,

$$\text{MSE}_m(t) = \frac{1}{n} \sum_{i=1}^n \left(m_i^{\text{GMsg}}(x, t) - m_i^{\text{grid}}(x, t) \right)^2. \quad (117)$$

For score vectors,

$$\text{MSE}_s(t) = \frac{1}{n} \sum_{i=1}^n \left(s_i^{\text{GMsg}}(x, t) - s_i^{\text{grid}}(x, t) \right)^2. \quad (118)$$

The relative score error is

$$\text{RelErr}_s(t) = \frac{\|s^{\text{GMsg}}(x, t) - s^{\text{grid}}(x, t)\|_2}{\|s^{\text{grid}}(x, t)\|_2}. \quad (119)$$

The cosine similarity is

$$\text{Cos}(t) = \frac{\langle s^{\text{GMsg}}(x, t), s^{\text{grid}}(x, t) \rangle}{\|s^{\text{GMsg}}(x, t)\|_2 \|s^{\text{grid}}(x, t)\|_2}. \quad (120)$$

The code also verifies the residual of the identity

$$s^{\text{GMsg}} - s^{\text{grid}} = \frac{\alpha t}{\Delta_t} (m^{\text{GMsg}} - m^{\text{grid}}), \quad (121)$$

which should be at numerical precision.

14 Numerical results from the report run

The report run uses:

Parameter	Value
Chain length	$n = 50$
Correlation	$\rho = 0.85$
Grid interval	$[-12, 12]$
Validation grid sizes	151, 301, 601
Laplace convergence grids	301 vs 601
Reference grid size	601
Trials per time	50
Diffusion times	0.08, 0.12, 0.18, 0.27, 0.40, 0.60, 0.90, 1.30, 1.80, 2.40
Boundary-mass tolerance	10^{-6} over 3 edge cells

The grid sizes correspond to spacings 0.16, 0.08, 0.04, the same resolutions used in the first version of this report, but on a wider interval. The widening was prompted by the boundary-mass diagnostic described in Section 10.5: on $[-8, 8]$ the worst observed boundary mass was 7.4×10^{-7} , uncomfortably close to tolerance, whereas on $[-12, 12]$ at equal resolution it falls to 5.3×10^{-10} .

14.1 Gaussian validation of grid BP

Figure 1 compares grid BP with the exact Gaussian precision-matrix score. This validates the grid procedure in a setting where the true score is known analytically.

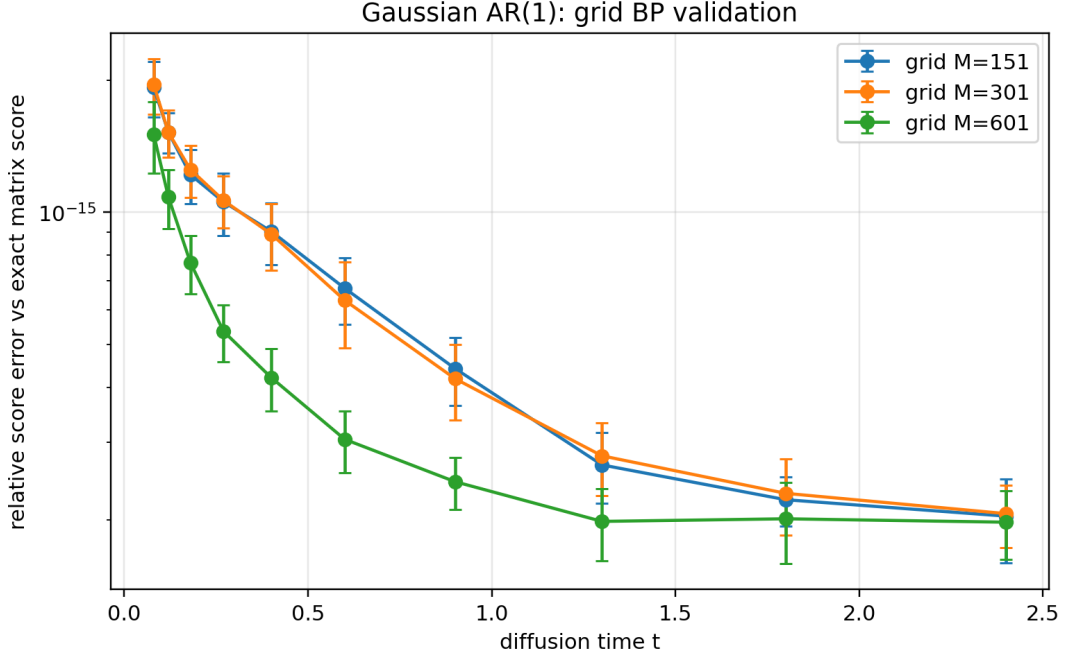


Figure 1: Gaussian AR(1) validation: relative error between grid BP score and exact precision-matrix score. The error is extremely small for the chosen interval and grid sizes, confirming that grid BP is a reliable numerical reference in this controlled case.

Figure 2 checks Gaussian message closure in the Gaussian case, where it should be exact. The information-form recursion of Section 11 sits at 2×10^{-16} uniformly in t , confirming that the Gaussian family really is closed under the BP updates and that nothing is lost by storing only two scalars per message.

The same figure overlays the discarded grid projection of Section 11.2. Although structurally it makes the same Gaussian assumption, its error grows by fifteen orders of magnitude across the time range, reaching 1.8×10^{-1} at $t = 2.4$, and is essentially independent of M — confirming that the failure comes from the width of the truncation rather than from its resolution.

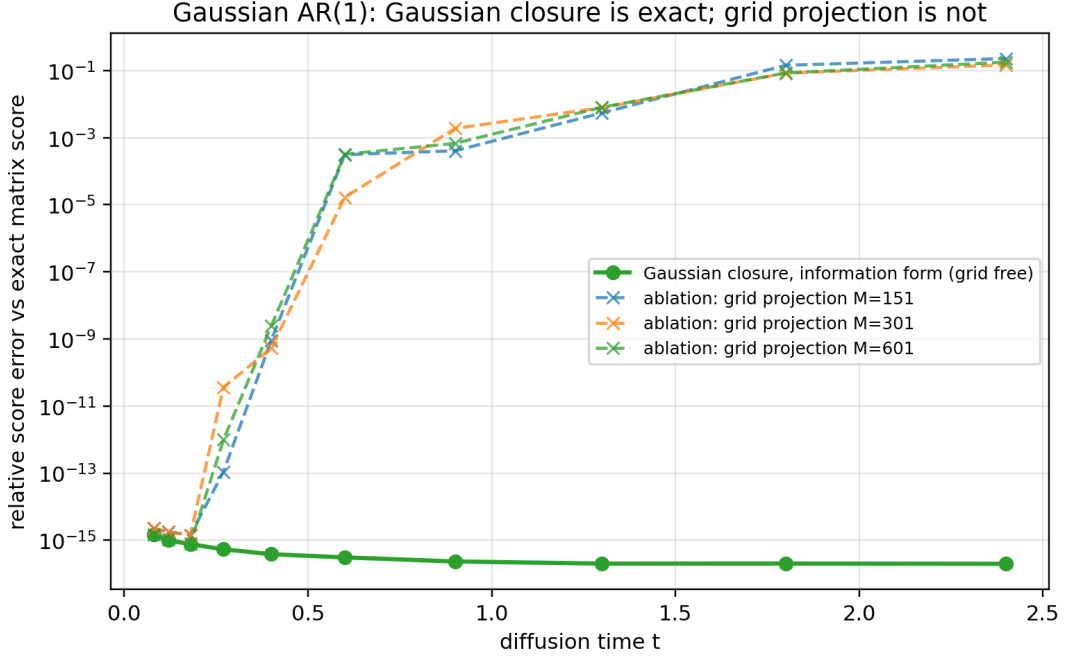


Figure 2: Gaussian AR(1): Gaussian message closure against the exact matrix score. The grid-free information-form recursion (solid) is exact to machine precision at every diffusion time. The discarded grid projection (dashed) is shown as an ablation; its error grows with t because $\alpha_t \rightarrow 0$ flattens the messages and the moment matching then measures the width of the grid rather than the shape of the message. The three dashed curves nearly coincide, showing the failure is insensitive to M .

14.2 Laplace grid convergence

In the Laplace case there is no precision-matrix score. Figure 3 compares grid BP with $M = 201$ and $M = 401$. This estimates the grid discretization error of the reference method.

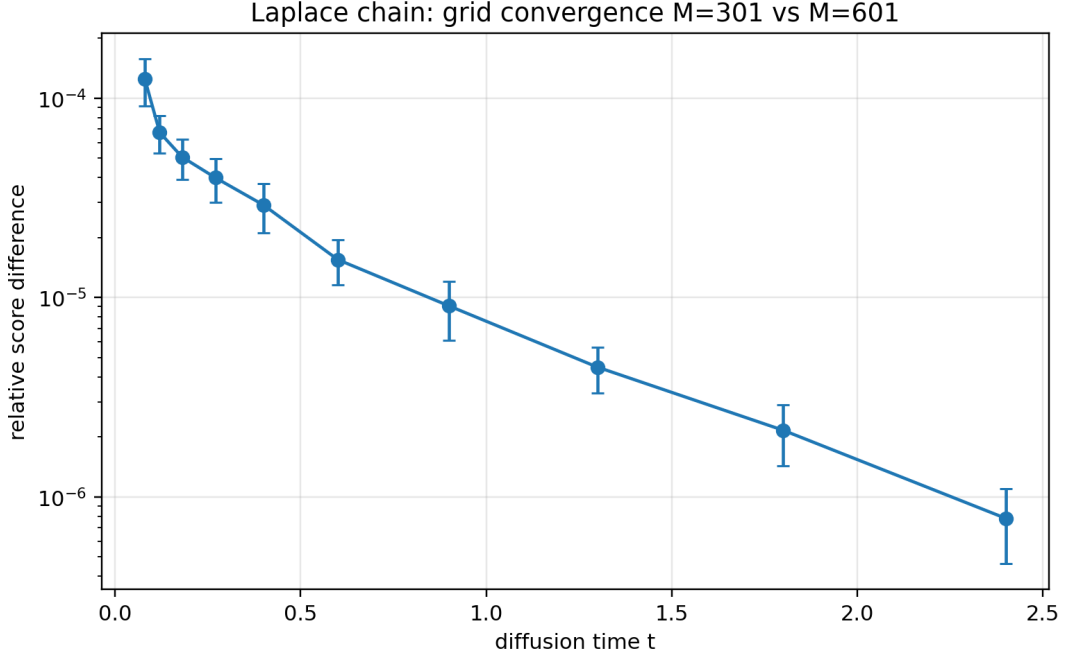


Figure 3: Laplace chain grid convergence: relative score difference between coarse grid $M = 201$ and fine grid $M = 401$. This is our proxy for grid discretization error in the non-Gaussian case.

Representative values from the Laplace grid-convergence CSV are:

t	Relative score difference	Posterior mean MSE	Score cosine
0.08	1.25×10^{-4}	1.23×10^{-9}	0.99999999
0.40	2.91×10^{-5}	7.92×10^{-10}	0.9999999996
1.30	4.48×10^{-6}	2.37×10^{-10}	0.99999999999
2.40	7.81×10^{-7}	8.67×10^{-11}	1.0000000000

The key interpretation is that the grid-discretization error is much smaller than the Gaussian-message approximation error measured below, at least for the report settings.

14.3 Gaussian-message approximation error in the Laplace chain

By Proposition 2 the Gaussian closure applied to this chain returns exactly the linear denoiser, so the quantities plotted here are the excess error of the best linear estimator relative to the exact one. The identity is confirmed numerically: the relative gap between the closure output and $\alpha_t \Sigma_0 \Sigma_t^{-1} x$ is at most 8.5×10^{-16} across the whole time range.

Figure 4 shows posterior-mean MSE. Figure 5 shows relative score error. Figure 6 shows score cosine similarity.

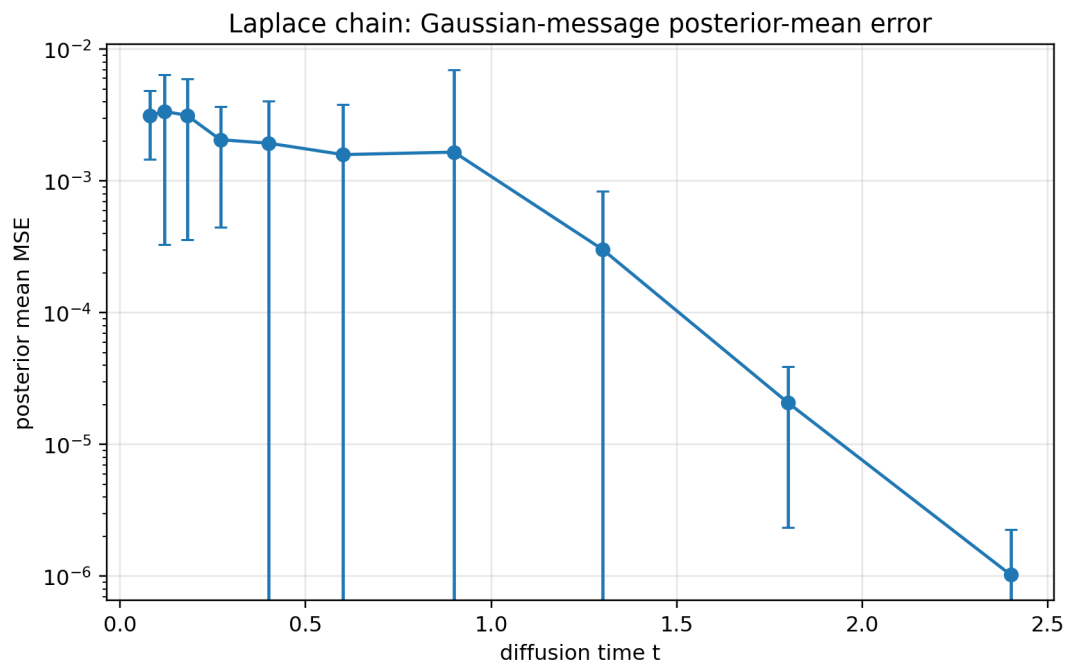


Figure 4: Laplace chain: posterior-mean MSE between Gaussian message closure and reference grid BP.

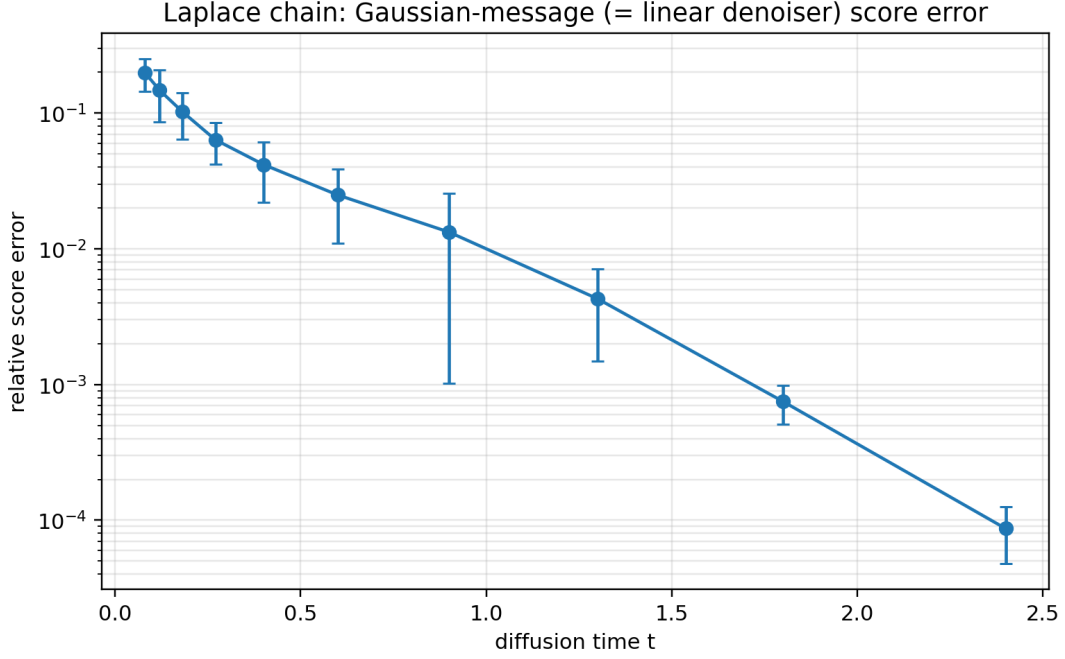


Figure 5: Laplace chain: relative score error of Gaussian message closure compared with reference grid BP. The error decreases monotonically with diffusion time: as t grows the posterior is increasingly dominated by the Gaussian likelihood and the linear estimator becomes correspondingly better. The non-monotonic behaviour reported in the first version of this note was an artefact of the grid projection discussed in Section 11.2.

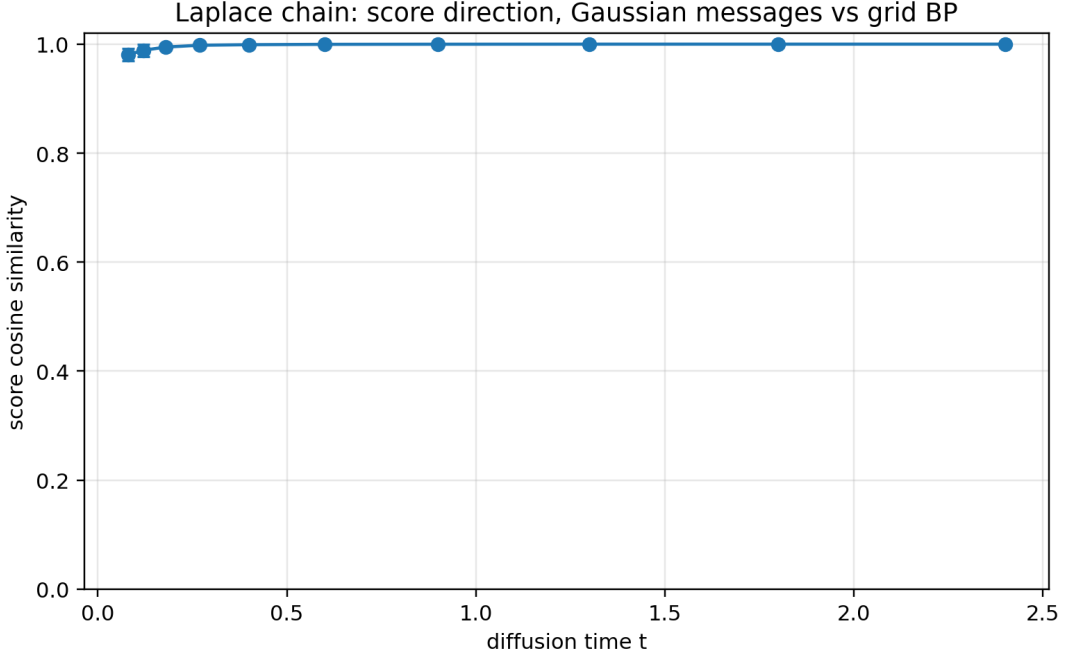


Figure 6: Laplace chain: cosine similarity between Gaussian-message score and reference grid-BP score. Even when the norm error is non-negligible, the direction remains close at all diffusion times.

Representative values, over 50 trials per time:

t	Posterior mean MSE	Relative score error	Score cosine	Closure vs LMMSE
0.08	3.15×10^{-3}	0.199	0.9806	8.5×10^{-16}
0.12	3.37×10^{-3}	0.147	0.9887	6.8×10^{-16}
0.40	1.94×10^{-3}	0.0418	0.9990	4.9×10^{-16}
0.90	1.66×10^{-3}	0.0133	0.9999	3.6×10^{-16}
1.80	2.07×10^{-5}	7.50×10^{-4}	1.0000	4.0×10^{-16}
2.40	1.03×10^{-6}	8.68×10^{-5}	1.0000	6.2×10^{-16}

Two features of this table are worth recording. First, both error measures now decrease monotonically in t ; the large values previously reported at $t = 1.80$ and $t = 2.40$ were entirely due to the grid projection. Second, the largest discrepancy occurs at *small* diffusion times, where the posterior is sharp and the non-Gaussian shape of the Laplace innovations is least washed out by the observation noise.

It does not follow that small times are the ones that matter for generation. The score error and the sampling error are different quantities, related by the whole backward trajectory rather than pointwise, and the accumulated effect of a score error at time t is not proportional to its size. Settling this requires running the reverse process with the exact and approximate scores and comparing the generated distributions; that is the natural next experiment and it is listed first in Section 16.

14.4 Example beliefs

Figure 7 compares a posterior belief at one site. This is the most concrete way to see what Gaussian projection does: it replaces the reference grid posterior shape by a Gaussian-message-

induced approximation.

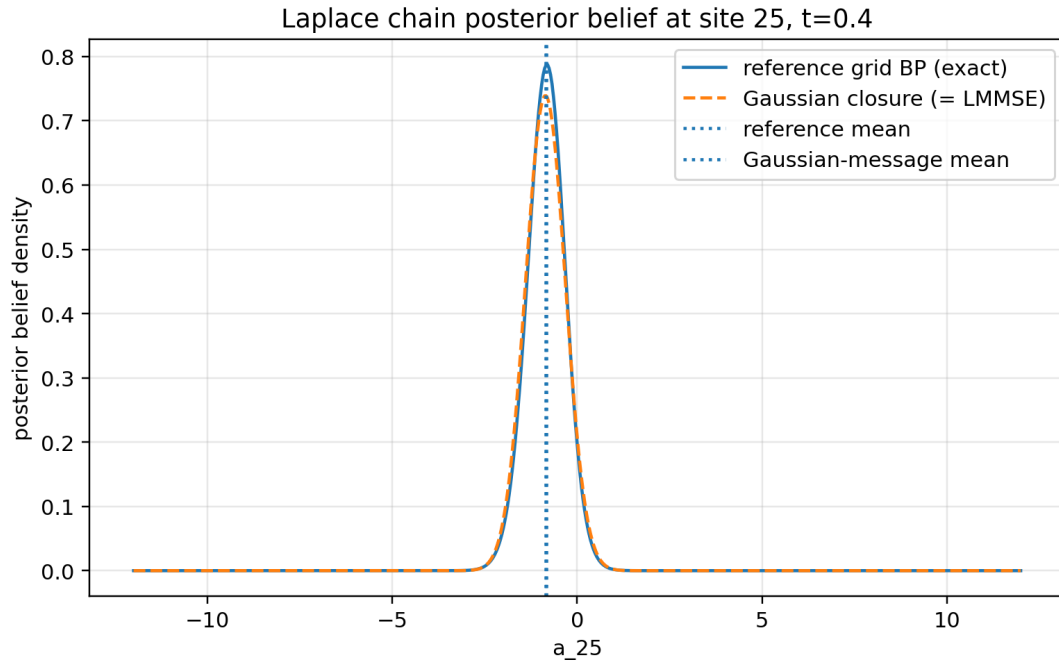


Figure 7: Example posterior marginal in the Laplace chain. The reference grid BP belief is compared with the Gaussian closure belief, which by Proposition 2 is the linear denoiser. The vertical lines show the corresponding posterior means.

Figure 8 shows an example Gaussian validation score component comparison.

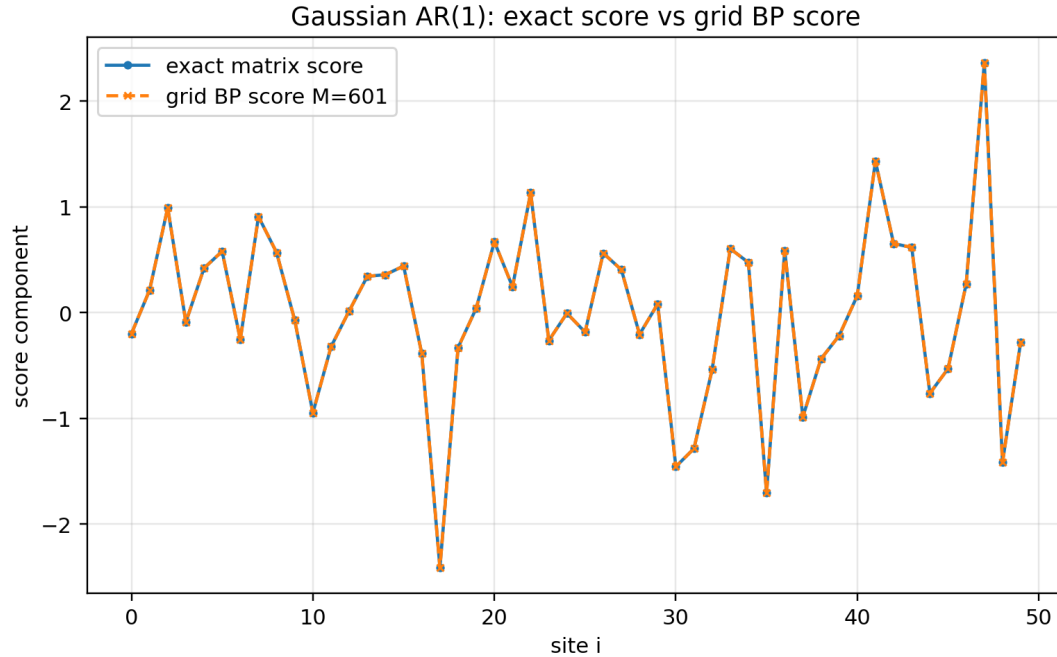


Figure 8: Gaussian AR(1) example: exact precision-matrix score versus grid BP score.

15 Code explanation

The Python script is

`code/bp_general_markov_diffusion_experiments.py`.

It is organized in layers matching the mathematics.

15.1 Configuration

The dataclass `ExperimentConfig` stores all experimental parameters:

```
@dataclass
class ExperimentConfig:
    n: int = 50
    rho: float = 0.85
    grid_min: float = -8.0
    grid_max: float = 8.0
    validation_grid_sizes: tuple[int, ...] = (101, 201, 401)
    convergence_coarse_grid_size: int = 201
    ref_grid_size: int = 401
    n_trials: int = 6
    seed: int = 11
    output_dir: str = "bp_markov_diffusion_outputs"
    t_values: tuple[float, ...] = (...)
```

This makes it easy to rerun heavier experiments by changing `n_trials`, `ref_grid_size`, or the grid interval.

15.2 Densities and numerical normalization

The functions `normal_pdf` and `laplace_pdf` implement the Gaussian and Laplace densities. The function `trapezoid_weights` builds quadrature weights. The function `normalize_density` normalizes grid values as a density:

```
def normalize_density(values: np.ndarray, weights: np.ndarray) -> np.ndarray:
    values = np.maximum(np.asarray(values, dtype=float), 0.0)
    mass = float(np.sum(values * weights))
    if not np.isfinite(mass) or mass <= 0.0:
        raise FloatingPointError("Cannot normalize density.")
    return values / mass
```

Normalization after each message update is numerically important. BP messages are only defined up to multiplicative constants, so normalization does not change beliefs, but it prevents underflow/overflow over long chains.

15.3 Transition matrices

The transition matrix is stored as

$$K[\text{out}, \text{in}] = p(a_{\text{out}} \mid a_{\text{in}}). \quad (122)$$

This convention makes the forward update a matrix-vector product:

$$L_{i+1} = K [(L_i \odot \ell_i) \odot w]. \quad (123)$$

The code is:

```
def build_transition_matrix(grid, rho, prior):
    q = 1.0 - rho ** 2
    out = grid[:, None]
    inn = grid[None, :]
    if prior == "gaussian":
        return normal_pdf(out, rho * inn, q)
    if prior == "laplace":
        return laplace_pdf(out, rho * inn, np.sqrt(q / 2.0))
```

15.4 Grid BP implementation

The core of `grid_bp` mirrors equations (87) and (90):

```
L[0] = normalize_density(normal_pdf(grid, 0.0, 1.0), weights)
for i in range(n - 1):
    incoming = L[i] * ell[i] * weights
    L[i + 1] = K @ incoming
    L[i + 1] = normalize_density(L[i + 1], weights)

R[-1] = np.ones(m)
for i in range(n - 1, 0, -1):
    incoming = R[i] * ell[i] * weights
    R[i - 1] = K.T @ incoming
    R[i - 1] = normalize_density(R[i - 1], weights)
```

After the two passes, beliefs and means are computed as:

```
b = normalize_density(L[i] * ell[i] * R[i], weights)
means[i] = np.sum(grid * b * weights)
```

This is exactly the discrete quadrature version of the exact BP belief formula.

15.5 Gaussian message closure implementation

The function `gaussian_closure_bp` takes only $(x, \rho, q, \alpha_t, \Delta_t)$ — no grid, no transition matrix — and propagates the two information scalars directly:

```
lam_t = lam_L[i] + lam_obs # multiply by the local likelihood
h_t = h_L[i] + h_obs[i]
mu_t, v_t = h_t / lam_t, 1.0 / lam_t
v_out = q + rho ** 2 * v_t # push through the transition
lam_L[i + 1] = 1.0 / v_out
h_L[i + 1] = rho * mu_t / v_out
```

The flat terminal right message is represented by zero precision, which is the exact encoding of infinite variance and costs nothing:

```
lam_R[-1], h_R[-1] = 0.0, 0.0
```

Beliefs are then formed by adding information parameters, and the posterior mean is h_i/λ_i .

15.6 The grid-projection ablation

`gaussian_projected_bp_grid` retains the earlier implementation, in which the same Gaussian assumption is routed through the grid:

```
L_values = gaussian_message_values(grid, L_mean[i], L_var[i])
incoming = L_values * ell[i] * weights
outgoing = K @ incoming
L_mean[i + 1], L_var[i + 1] = density_moments(outgoing, grid, weights)
```

with the flat terminal message written as

```
R_mean[-1] = 0.0
R_var[-1] = np.inf
```

The helper `gaussian_message_values` returns a vector of ones when the variance is infinite, and `density_moments` then assigns that constant the variance of the truncation interval rather than infinity. This single line is the origin of the failure analysed in Section 11.2.

15.7 Exact Gaussian score functions

For validation, the code builds

$$\Sigma_0(i, j) = \rho^{|i-j|}, \quad \Sigma_t = \alpha_t^2 \Sigma_0 + \Delta_t I, \quad (124)$$

and computes

$$s = -\Sigma_t^{-1} x \quad (125)$$

using `np.linalg.solve`:

```
def gaussian_precision_score(x, rho, alpha, delta):
    sigma0 = ar1_covariance(len(x), rho)
    sigmat = alpha ** 2 * sigma0 + delta * np.eye(len(x))
    return -np.linalg.solve(sigmat, x)
```

Using `solve` is numerically preferable to explicitly computing Σ_t^{-1} .

15.8 Experiment runners

The script has three experiment runners:

1. `run_gaussian_grid_validation`: compares grid BP, Gaussian closure and the grid-projection ablation against the exact matrix score.
2. `run_laplace_grid_convergence`: compares coarse-grid and fine-grid BP in the Laplace chain.
3. `run_laplace_gaussian_message_error`: compares Gaussian closure against reference grid BP in the Laplace chain, and verifies the LMMSE identity.

The final function `run_all` executes all experiments, writes CSV files, and generates plots.

15.9 How to run

From the root folder of the delivered archive:

```
python code/bp_general_markov_diffusion_experiments.py --mode quick
```

for a fast smoke test, or

```
python code/bp_general_markov_diffusion_experiments.py --mode report
```

for the report-quality run. To increase experiment weight:

```
python code/bp_general_markov_diffusion_experiments.py \
    --mode report \
    --n-trials 50 \
    --ref-grid-size 801 \
    --output-dir outputs_heavy
```

The cost grows approximately as $O(nM^2)$, so increasing M is much more expensive than increasing the number of trials.

16 Interpretation and next steps

The clean conclusion is:

BP gives the exact score for Markov-chain priors under OU noising.

The hard part is not the score formula. The hard part is continuous message representation.

The Gaussian approximation should therefore be studied as a message-representation approximation. In the Laplace-innovation experiment, the grid error is tiny compared with the Gaussian-message error, so the observed discrepancy primarily reflects Gaussian closure error rather than grid error.

Natural next steps are:

1. Increase trials and grid size to reduce Monte Carlo variability.
2. Explore different ρ , because stronger correlations make messages more informative and potentially more non-Gaussian.
3. Compare Laplace, Student, Gaussian-mixture, and bounded-support innovations.
4. Add Gaussian-mixture message approximations to see how many mixture components are needed to beat single-Gaussian closure.
5. Study whether neural message approximators help only when they preserve the local BP update structure.

The proposed thesis direction is not that neural networks are unnecessary in general. In real diffusion models, the clean prior p_0 is unknown and must be learned from data. The precise claim is more structural:

Score learning can be reformulated as posterior denoising. For Markov-chain priors, posterior denoising is exact BP. When exact continuous BP is not computationally closed, one must approximate the messages. Gaussian messages are the first approximation to test, and their error can be measured directly through the posterior-mean/score identity.

A Files in the delivered folder

The delivered folder contains:

File	Purpose
report/bp_markov_diffusion_gaussian_approx.pdf	this report as PDF
report/bp_markov_diffusion_gaussian_approx.tex	LaTeX source
code/bp_general_markov_diffusion_experiments.py	full experiment script
code/bp_general_markov_diffusion_experiments.ipynb	executable/exploratory notebook
outputs/*.csv	numerical experiment tables
outputs/*.png	plots used in the report

B Source references

References

- [1] G. Mantovani, *Belief Propagation for the Gaussian AR(1) Diffusion Model: A complete derivation and numerical comparison with the precision-matrix score*, internal project note, 2026.
- [2] M. Mezard and A. Montanari, *Information, Physics, and Computation*, Oxford University Press, 2009. Relevant background: factor graphs and belief propagation on trees.
- [3] F. Krzakala and L. Zdeborova, *Statistical Physics Methods in Optimization and Machine Learning*, lecture notes, 2024. Relevant background: BP equations, sparse graphs, and the Bethe perspective.
- [4] G. Biroli and M. Mezard, *Generative diffusion in very large dimensions*, arXiv:2306.03518, 2023.

- [5] G. Biroli, T. Bonnaire, V. de Bortoli, and M. Mezard, *Dynamical Regimes of Diffusion Models*, arXiv:2402.18491, 2024.
- [6] S. Mei, *U-Nets as Belief Propagation: Efficient Classification, Denoising, and Diffusion in Generative Hierarchical Models*, arXiv:2404.18444, 2024.
- [7] G. Genovese and A. Piana, *Derivation of the AMP Equations from Belief Propagation for the ℓ_2 Minimisation Problem*, arXiv:2602.15191, 2026.